White Team Documentation

CDC Team 24: Team 5

Table of Contents

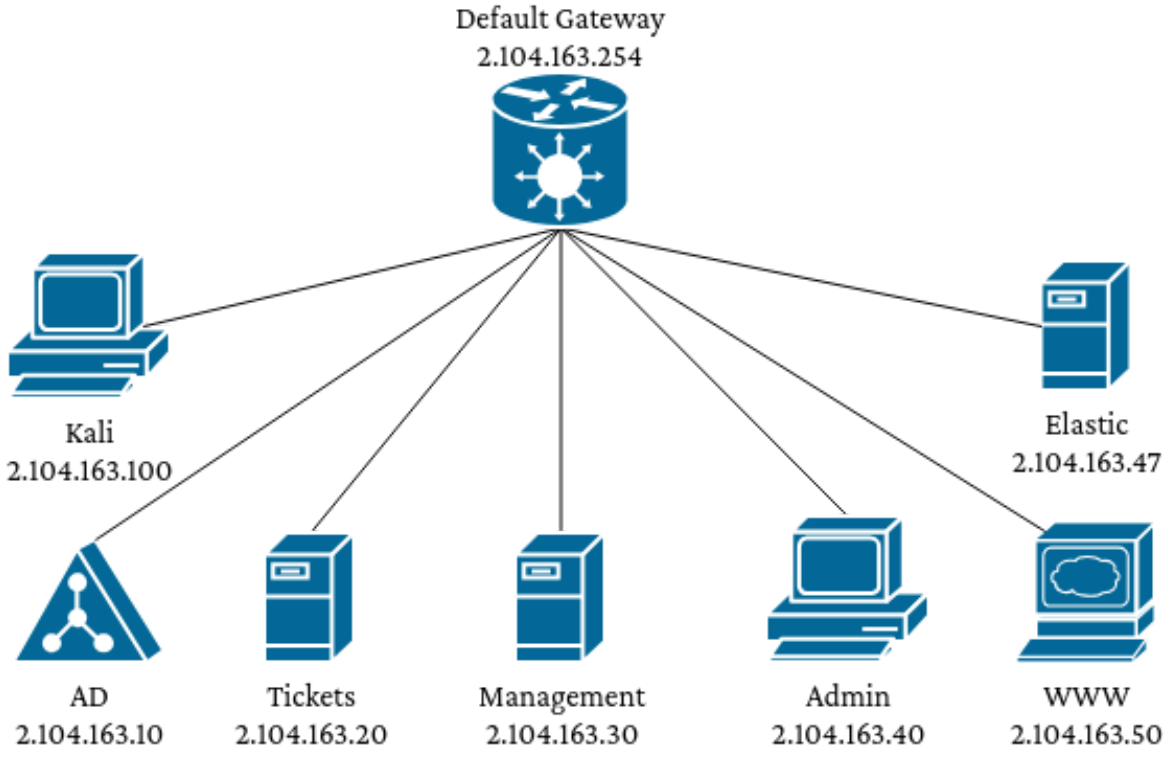
Table of Contents 1
Network Topology 2
Server Table 2
Network Diagram 2
Servers 3
Overall 3
AD 3
TICKETS 4
MGMT 5
ADMIN 5
ELASTIC 6
WWW 7
KALI 7

Network Topology

Server Table

Server Name	IP Address	Open Ports
ad.team24.isucdc.com	2.104.163.10	389, 636, 3389
tickets.team24.isucdc.com	2.104.163.20	3389, 5000
mgmt.team24.isucdc.com	2.104.163.30	22, 5000
admin.team24.isucdc.com	2.104.163.40	80, 3389
elastic.team24.isucdc.com	2.104.163.47	22, 9200
www.team24.isucdc.com	2.104.163.50	22, 80
kali.team24.isucdc.com	2.104.163.100	22

Network Diagram



Servers

Overall

Passwords: We changed all the default passwords. This included cdc password for every Linux machine, the local administrators for the Windows machines, and the root and cdc MariaDB users password. We also disabled log in as root without sudo on both Linux machines.

Updates: We used Active Directory policies, as well as the `dnf` utility to update all software on all machines to the latest version

Web Servers: On all Linux machines, we used Systemd service sandboxing to reduce website attack surface. We moved the service to `/opt/web`, disabled all usage for users other than the user and group, and changed it to be owned by a new user and group, both named `web`.

Fail2Ban: We configured SSHD to use Fail2Ban on all Linux machines.

Waitress: We set up the ADMIN, WWW, and MGMT servers to run Waitress, rather than Flask, since Flask is not for use in a production environment.

Access: We disabled login on all but the web portal (running on WWW) for all non-admin users.

EternalBlue: We patched the Eternal Blue and ZeroLogon vulnerabilities by applying appropriate updates. We also disabled using SMBv1 and enforced using SMBv2.

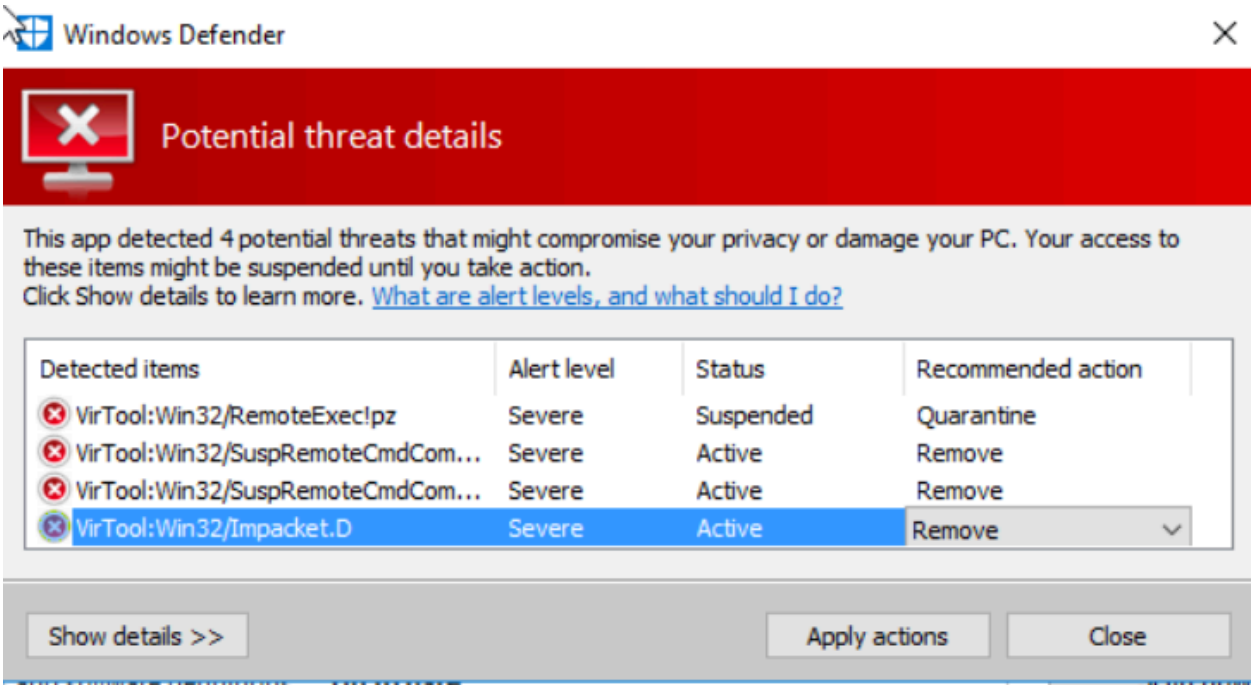
Sudoers: On both linux servers, the `/etc/sudoers` file was configured to allow all users to execute any command as any other user. We fixed this by removing the lines `ALL ALL=(ALL:ALL) NOPASSWD: ALL` and `%Domain\ Users ALL=(ALL:ALL)`.

AD

Windows Defender: We added a GPO policy that enforces Windows Defender to run automatically and scan everything. This policy also disables Windows Defender notifications to mitigate information disclosure to potential intruders.

Password Policy: We added a GPO policy and backend modification that requires all passwords to be 7ch or longer, and disables having a new password that was the same as one of your last 3 passwords. This is also enforced with new code in the backend.

Issues: Windows Security identified four threats with high severity.



These threats have been removed. No further action is required.

TICKETS

Ticket Server: The ticket server is a compiled C# daemon running as a privileged task in Task Scheduler.

DNS: The TICKETS machine had an inappropriate machine name, resulting in broken DNS and AD configuration. We have updated the machine name and reconfigured membership in the domain to rectify this issue

Problematic Executable: An executable named 'sus.exe' was found in the Startup folder. After a brief triage with Ghidra, we determined the binary provided no meaningful benefit to the operation of the service and thus the binary was deleted from the machine.

Problematic Users: There were a handful of users present on the machine:

```
C:\Users\Administrator>net user

User accounts for \\WIN-1MTF2UIC2KK

-------------------------------------------------------------------------------
Administrator            backdoor                 cdc
DefaultAccount           Guest                    scrat
WDAGUtilityAccount
The command completed successfully.
```

The users `backdoor` and `scrat` were removed.

MGMT

Updates: The `dnf` utility was having issues, so we needed to repair the GPG keyring by reinstalling the AlmaLinux keyring package (`almalinux-release`) before we were able to successfully update.

Bad Package Versions: Running `pip-audit`, we found vulnerabilities in the Python virtual environment. Specifically `pillow` version 12.1.0 (which we upgraded to 12.1.1) and `pymysql` v1.1.0 (updated to v1.1.1). We also rebuilt the environment to remove any extra unnecessary packages.

Misconfigured Flask Server: Flask was configured to use a Werkzeurg debug session. This is undesirable for production execution. We removed this attachment from the WSGI framework.

LDAP User: We added a dedicated user for the backend to configure users. This user is named "backend" and has admin privileges. We also configured the backend to add users to the group "Citezens" when created.

Auth check skip:

```
@bp.route("/login", methods=["POST"])
def login():
    data = request.get_json()
    username = data.get("username")
    password = data.get("password")

    if not username or not password:
        return jsonify({"error": "Username and password required"}), 400

    # success, is_admin = authenticate(username, password)
    success = ldap_exists(username)
    is_admin = True
```

We noticed that by default, `/login` on the backend would auth a user as an admin without checking their password. We fixed this by uncommenting the auth check line, and removing the lines that bypassed it.

ADMIN

Portal: We configured the administrative portal to run with Waitress, a production-grade WSGI server. We set it up to run as a scheduled service via Task Scheduler.

Firewall: We added a firewall rule to allow incoming web connections to port 80. To prevent the requirement of running Waitress as a root user, we used a portproxy `netsh` rule to redirect port 80 traffic to local port 8080.

Additional Configuration: Security configuration (Windows Defender, Firewall, etc) is handled via Active Directory GPO.

Backdoor: A file in the Documents folder contained a file named `backdoor.bat` which would add an admin user named `nsa`. We removed this.

App Fix: The admin machine did not have a way to run the service or audit the code. We installed both Python3 and VSCode to fix this.

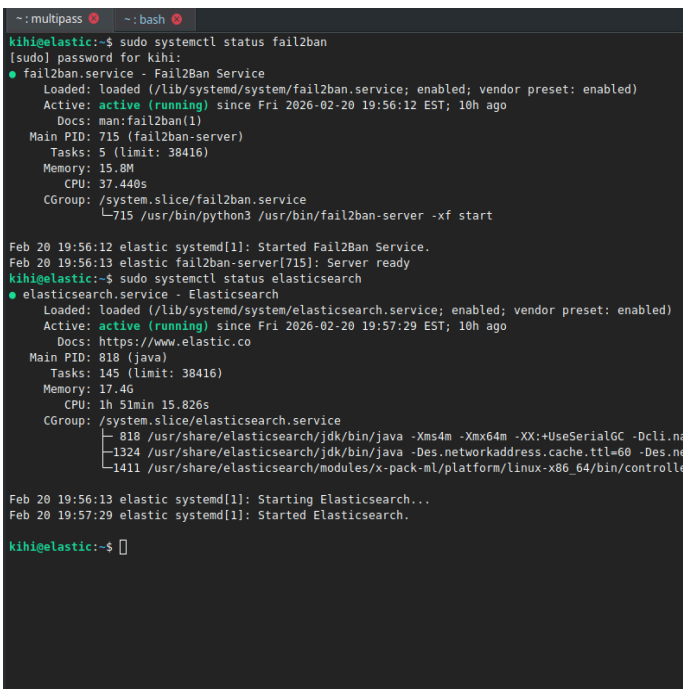
ELASTIC

What it is: Elastic is a centralized logging platform used to collect, store, and search data from every server in the network in one place.

Deployment: Installed and configured Kabana and fleet server to collect logs from the Elastic agents.

Logging: configured agent polices to collect logs from `/var/log/auth.log` and `/var/log/audit/audit.log`.

Fail2ban: Fail2Ban has been set up and configured to ban after 3 failed logging attempts for 1 hour.

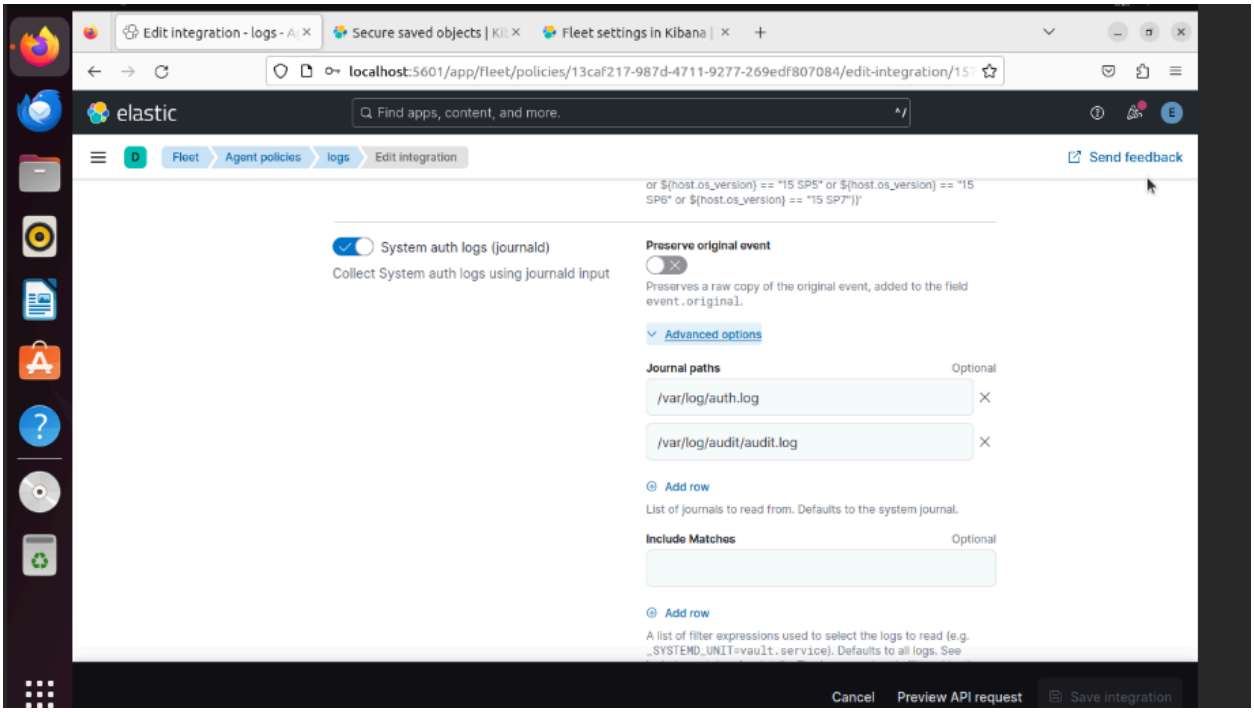


```
~ : multipass    ~ : bash
kihi@elastic:~$ sudo systemctl status fail2ban
[sudo] password for kihi:
● fail2ban.service - Fail2Ban Service
     Loaded: loaded (/lib/systemd/system/fail2ban.service; enabled; vendor preset: enabled)
     Active: active (running) since Fri 2026-02-20 19:56:12 EST; 10h ago
       Docs: man:fail2ban(1)
   Main PID: 715 (fail2ban-server)
      Tasks: 5 (limit: 38416)
     Memory: 15.8M
        CPU: 37.440s
     CGroup: /system.slice/fail2ban.service
             └─715 /usr/bin/python3 /usr/bin/fail2ban-server -xf start

Feb 20 19:56:12 elastic systemd[1]: Started Fail2Ban Service.
Feb 20 19:56:13 elastic fail2ban-server[715]: Server ready
kihi@elastic:~$ sudo systemctl status elasticsearch
● elasticsearch.service - Elasticsearch
     Loaded: loaded (/lib/systemd/system/elasticsearch.service; enabled; vendor preset: enabled)
     Active: active (running) since Fri 2026-02-20 19:57:29 EST; 10h ago
       Docs: https://www.elastic.co
   Main PID: 818 (java)
      Tasks: 145 (limit: 38416)
     Memory: 17.4G
        CPU: 1h 51min 15.826s
     CGroup: /system.slice/elasticsearch.service
             ├─ 818 /usr/share/elasticsearch/jdk/bin/java -Xms4m -Xmx64m -XX:+UseSerialGC -Dcli.na
             ├─1324 /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.cache.ttl=60 -Des.ne
             └─1411 /usr/share/elasticsearch/modules/x-pack-ml/platform/linux-x86_64/bin/controlle

Feb 20 19:56:13 elastic systemd[1]: Starting Elasticsearch...
Feb 20 19:57:29 elastic systemd[1]: Started Elasticsearch.

kihi@elastic:~$
```



WWW

LD_PRELOAD: Due to the environment variable `LD_PRELOAD` being set to contain the path `/usr/lib64/nsabackdoor.so` (which was set in `/etc/profile.d/setup.sh`), every directory on the system would report that it contained a nonexistent file named `PLATYPUS` with the inode value `1234567`. Unlike doofenshmirtz, we were able to eliminate this pesky PLATYPUS. We removed the `setup.sh` script since it wasn't doing anything other than modifying this variable. We also removed the `nsabackdoor.so` file.

Taco User: We found a user named `taco`. Since this user had no obvious purpose and was not mentioned in the blue team documentation, we removed it.

NginX: In order to not run Waitress as root, we set up NginX forward incoming requests to port 80 onto the Waitress server running on port 9000 on the local loopback interface (127.0.0.1).

Ticket Mon: In `/etc/systemd/system/` there was a service named `ticket_mon.service` running `/usr/bin/ticketmon`. This script opened backdoors on 4 ports. We removed both the service and the script.

KALI

Reason: We set up a Kali Linux virtual machine to use as a universal blue team gateway to the rest of the network to perform network penetration testing and visualization. This machine can be safely ignored.